Automated Search Lead Harvester

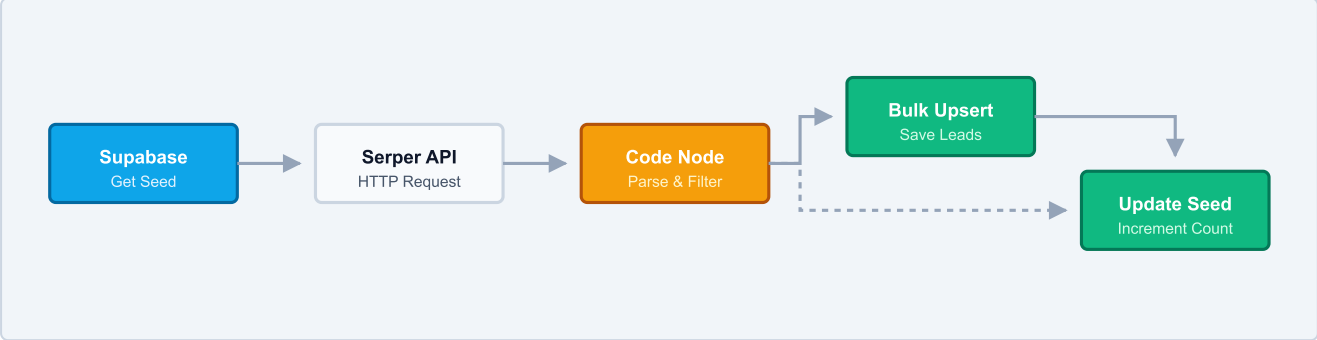
`n8n Workflow Architecture & Technical Specification` PRODUCTION

1. Executive Summary

This document outlines the architecture and implementation details for **Workflow 0**, an automated lead generation pipeline built within n8n. The system is designed to seamlessly retrieve geographic queue seeds from a Supabase PostgreSQL database, query Google Search via the Serper API for independent boutique storefronts, and process the raw JSON results. To ensure pristine database integrity, the pipeline employs a custom, sandbox-safe RegEx extraction engine that filters out directory sites and aggregates valid, unique commercial domains into the primary database.

2. Data Flow Architecture

The workflow executes sequentially, featuring isolated condition branches to ensure database operations are only triggered when viable leads are discovered.



Key Technical Achievements

- **Sandbox Isolation Bypass:** Overcame native n8n JavaScript execution limits by replacing standard `new URL()` constructors with regex domain extractors, preventing fatal `ReferenceError` crashes.
- **Intelligent Filtering:** Maintains an internal blacklist to instantly discard directories, social media hubs, and tourism portals (e.g., TripAdvisor, VisitBath).
- **Secure DB Transactions:** Avoids Supabase UI filter concatenation bugs by utilizing explicit `matchColumns: "id"` rules to prevent PostgreSQL "WHERE clause" missing parameters.

3. Component Specifications

The workflow is comprised of eight discrete operations. Data structures are strictly defined at each boundary to ensure seamless transition between the API extraction layer and the storage layer.

Node Name	Operation Type	Functionality & Configuration
`Get Next Seed`	Supabase (Read)	Queries `search_seeds` for 1 row where `status=eq.queued`.
`IF: Seed Found?`	Logic (Branch)	Validates payload. Halts execution if the query array is empty.
`Serper Search`	HTTP Request	Issues a POST to `google.serper.dev`. Appends geographic seed to standard string: "independent clothing store".
`Code: Filter`	JavaScript	Parses `rawOrganic` payload, applies Regex constraints, and removes duplicate hostnames via a JS `Set()`.
`IF: Valid Lead?`	Logic (Branch)	Routes array payload to Upsert if `leads_count >= 1`.
`Bulk Upsert`	Supabase (Insert)	Writes clean lead objects to `leads` table. Includes origin linkage via `origin_seed_id` mapping.
`Increment Counters`	Supabase (Update)	Updates the origin seed queue row. Configured with `executeOnce: true` to prevent counter inflation loops.

> ⚠ **Parallel Update Prevention**
> Because the Bulk Upsert node emits an array of items (one for each lead discovered), the downstream Increment node will attempt to execute multiple times. The toggle `executeOnce: true` explicitly forces n8n to flatten the trigger, correctly incrementing the queue counters exactly once per workflow execution.

4. Data Extraction & Cleansing Logic

The core logic of the pipeline relies on a customized JavaScript code node to translate raw Search Engine Result Pages (SERP) into clean business objects.

```
      // 1. Regex domain extraction without relying on URL global const match =
item.link.match(/^(?:https?:\/\/)?(?:www\.)?([^\/\?#:]+)/i); if (!match || !match[1])
continue; const host = match[1].toLowerCase(); // 2. Strict directory and marketplace
filtration const junkRoots = [ 'google.', 'bing.', 'yahoo.', 'reddit.com',
'facebook.com', 'instagram.com', 'tiktok.com', 'linkedin.com', 'amazon.', 'ebay.',
'etsy.com', 'yelp.', 'tripadvisor.', 'yell.com', 'visitbath.co.uk', 'soglos.com' ];
const isJunk = junkRoots.some(bad => host.includes(bad)) ||
host.startsWith('directory.'); if (!isJunk && !seenDomains.has(host) &&
host.includes('.')) { seenDomains.add(host); // 3. Title sanitization const rawTitle =
item.title || host.split('.')[0]; const cleanName = rawTitle.split(/[-|•–:\/]/)
[0].trim(); cleanLeads.push({ company_name: cleanName, website_url: `https://${host}`,
root_domain: host, origin_seed_id: seedId, status: 'pending' }); }
```

This implementation serves a dual purpose. By dropping standard URL parsing, it guarantees stability within n8n's isolated node environments. Secondly, the title sanitization splits raw meta titles at common delimiters (such as hyphens and pipes), isolating the commercial entity's true name from SEO keywords before committing to the database.

5. Database Persistence Structure

The pipeline relies on a robust schema design within Supabase to manage queue states and record provenance for all scraped lead data.

Table: `search_seeds`

Column Name	Data Type	Constraints & Defaults
`id`	`uuid`	`PRIMARY KEY`, `gen_random_uuid()`
`seed_type`	`text`	Must equal 'postal_district', 'town_cluster', or 'niche_expansion'.
`query_value`	`text`	`UNIQUE NOT NULL` (The geographic string appended to the search).
`times_searched`	`integer`	Defaults to `0`. Incremented securely using native matchColumns.
`leads_discovered`	`integer`	Defaults to `0`. Aggregates array length of discovered storefronts.
`status`	`text`	Must equal 'queued', 'processing', 'exhausted', or 'blacklisted'.
`last_searched_at`	`timestamptz`	Updated to `new Date().toISOString()` upon successful loop completion.

> ℹ **PostgREST Interaction Standard**
> When performing automated row updates in Supabase via external orchestrators (like n8n), relying on string-concatenated filter parameters (e.g., `id=eq.uuid`) is deprecated and prone to missing-variable exceptions. This architecture explicitly utilizes exact schema mapping via `matchColumns: "id"` to safely enforce PostgreSQL `WHERE` clauses and maintain primary key integrity.